

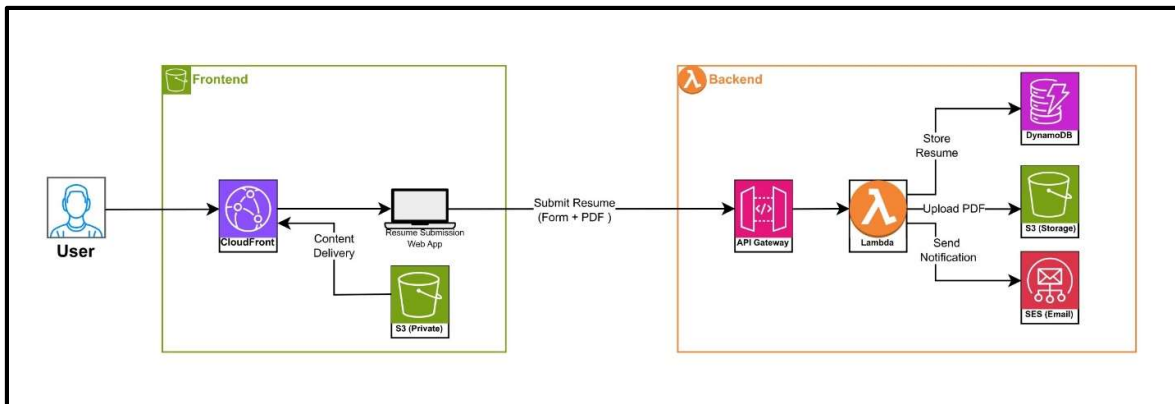
Serverless Resume Submission System — Full Documentation

• Overview

This project implements a serverless resume submission system using AWS services. Candidates upload resumes through a frontend UI, and the backend automatically stores files, logs metadata, and sends email notifications.

• Architecture Diagram

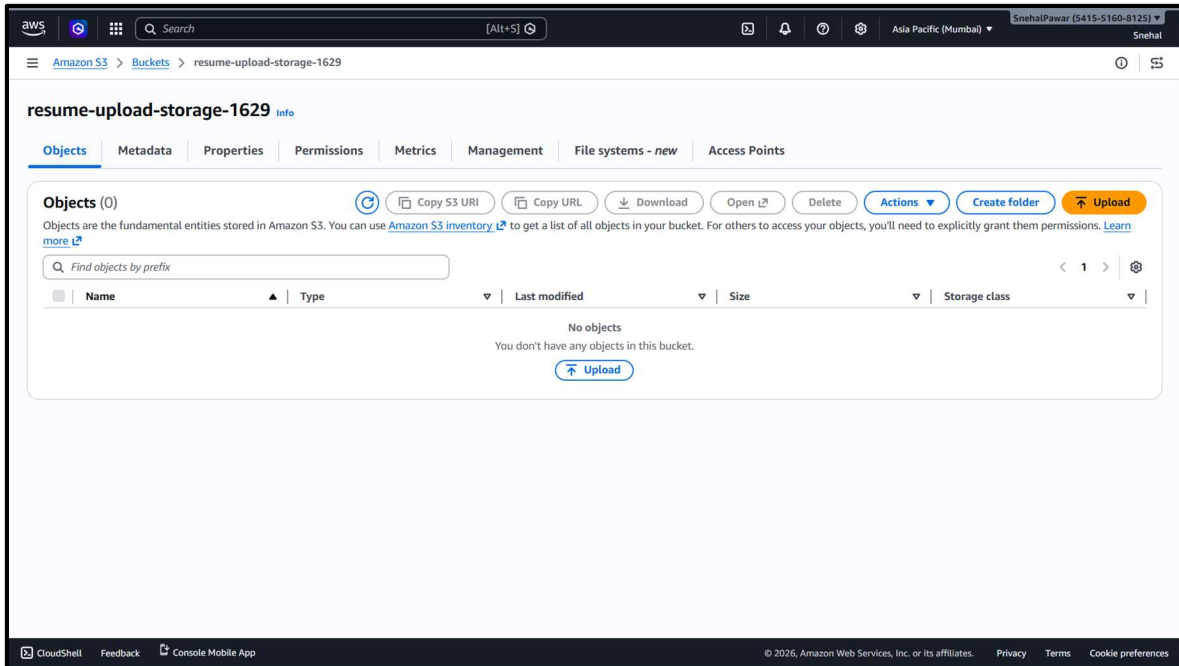
Frontend → API Gateway → Lambda → S3 → DynamoDB → SES



• Step 1 — Create Resume Storage Bucket

1. Go to AWS S3
2. Click Create bucket
3. Bucket name: resume-upload-storage
4. Keep Block Public Access ON
5. Create bucket

This bucket stores uploaded resumes.



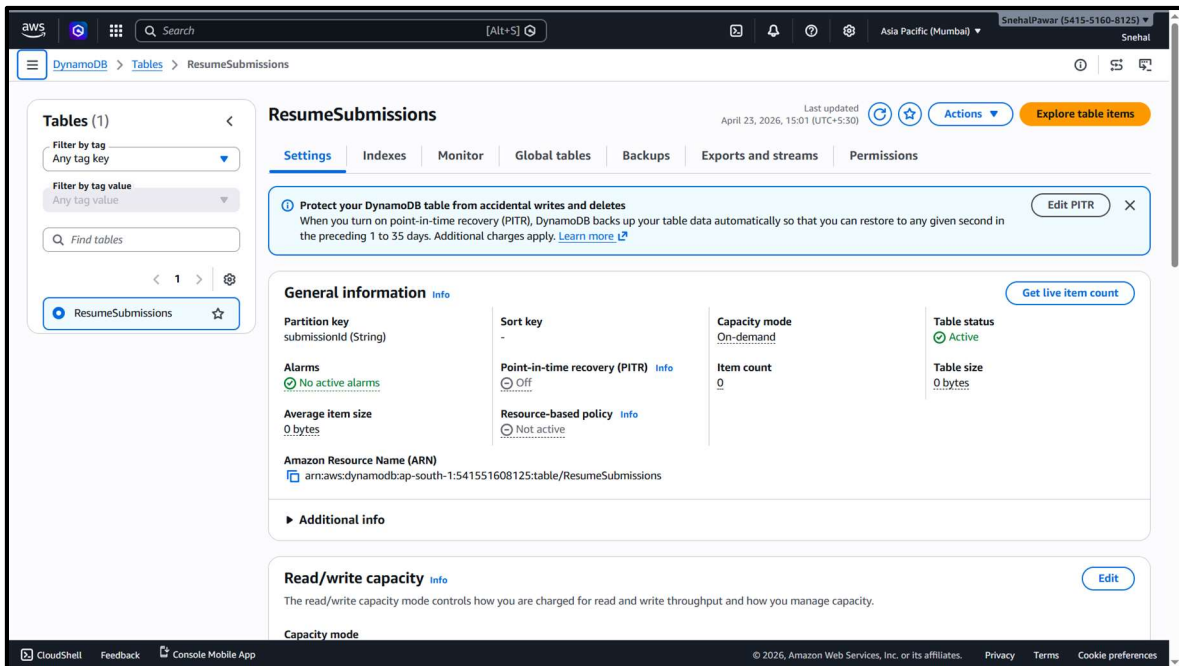
• Step 2 — Create DynamoDB Table

Service: DynamoDB

Table name: ResumeSubmissions

Partition key: submissionId (String)

Create table.



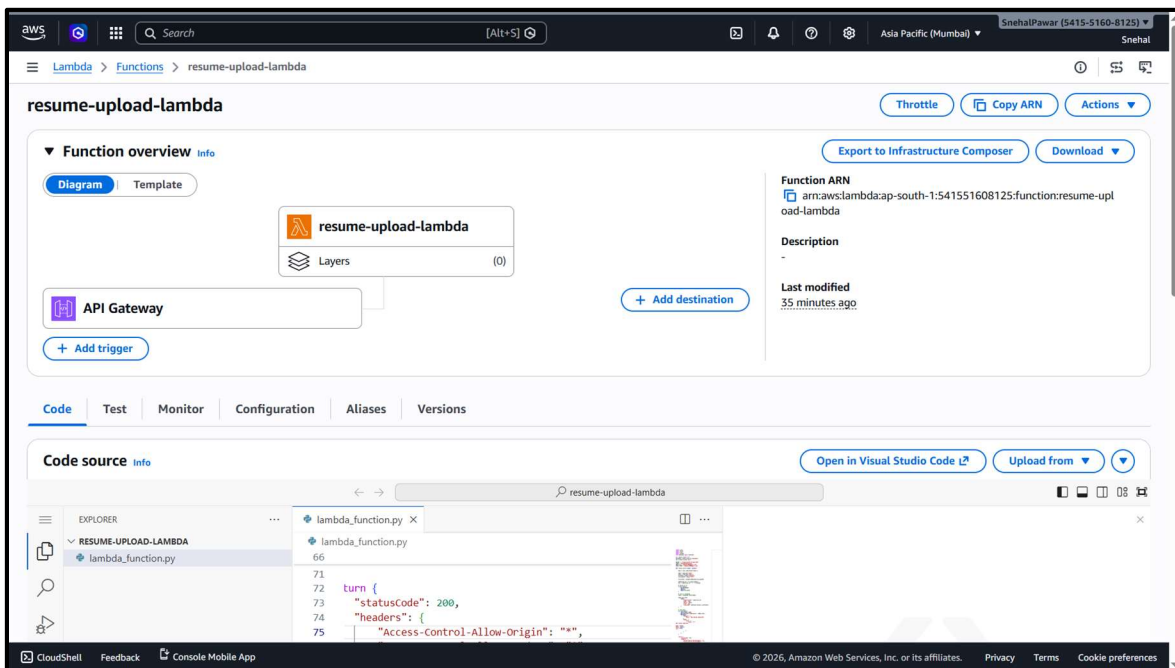
• Step 3 — Create Lambda Function

Runtime: Python 3.12

Function name: resume-upload-lambda

Purpose:

- Decode file
- Upload to S3
- Save metadata
- Send email



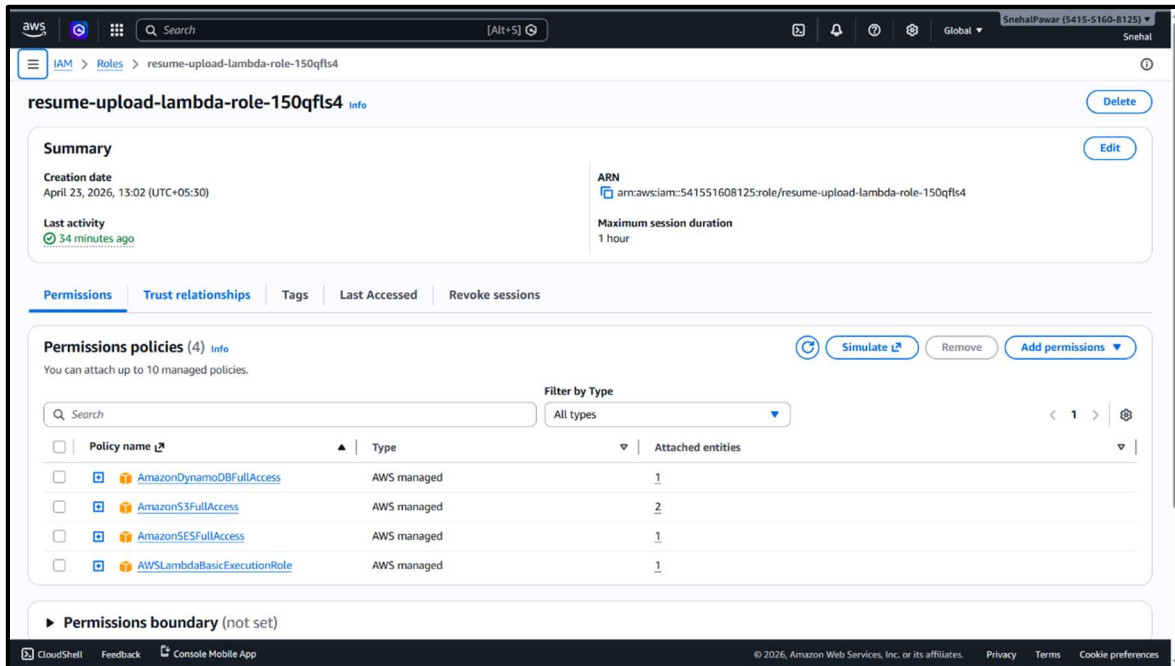
• Step 4 — Add IAM Permissions to Lambda

Attach policies:

AmazonS3FullAccess

AmazonDynamoDBFullAccess

AmazonSESEFullAccess



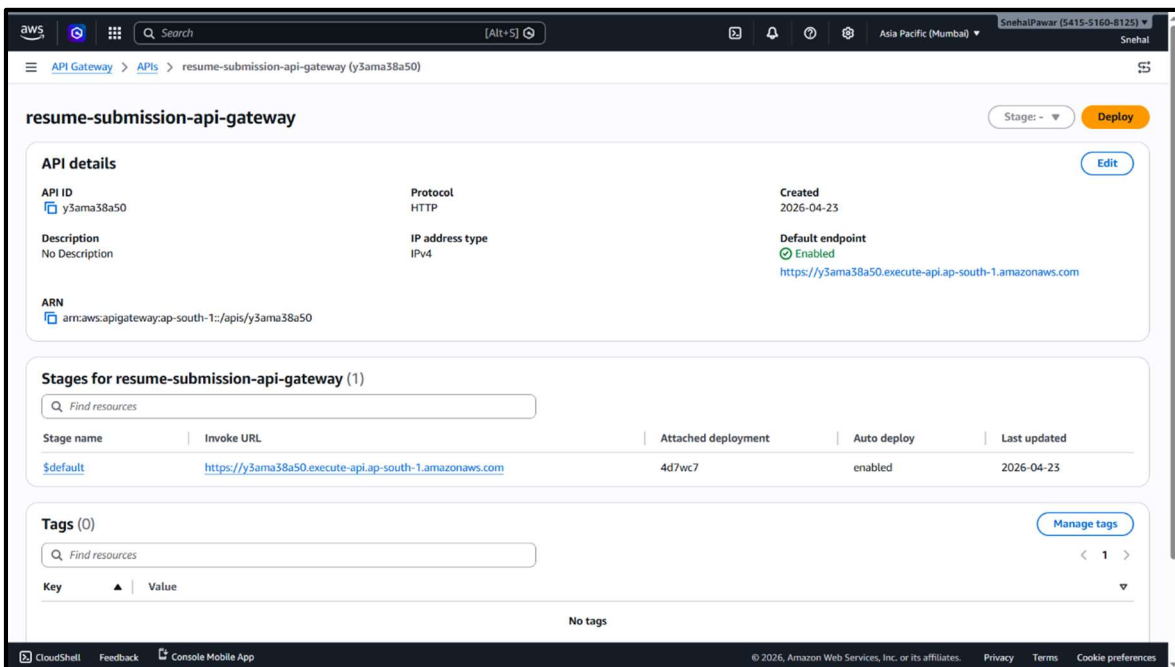
• Step 5 — Create API Gateway

Create HTTP API

Route: POST /submit

Integration: Lambda function

Enable CORS.



- **Step 6 — Build Frontend**

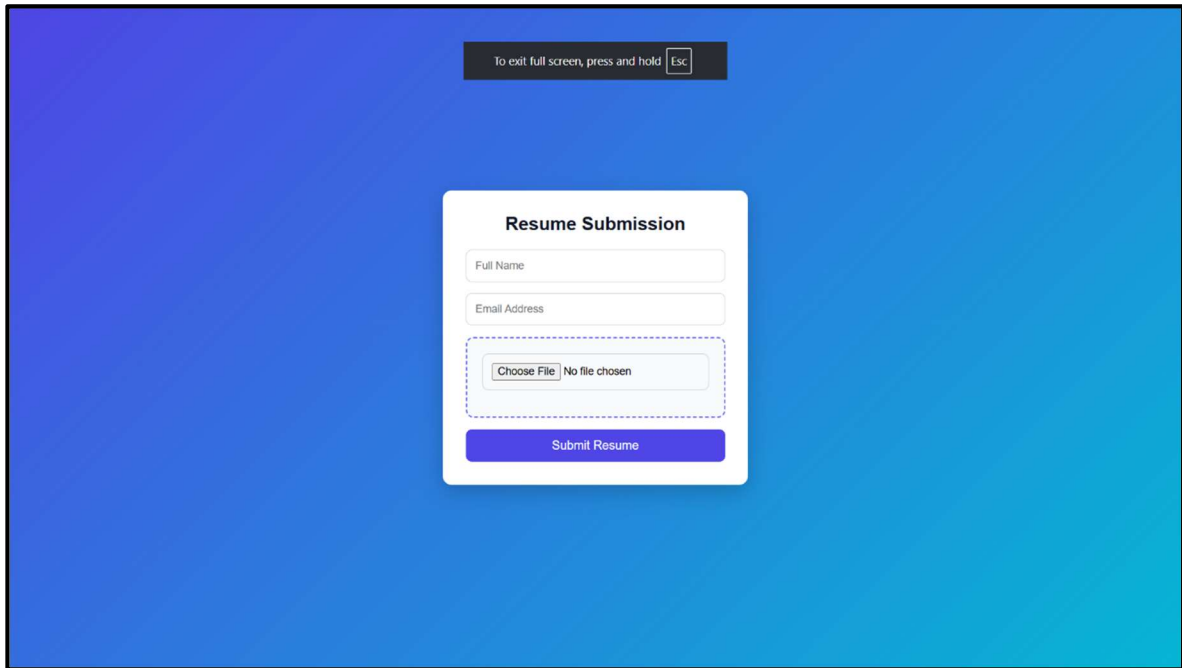
Fields:

Name

Email

File Upload

Frontend sends POST request to API Gateway.



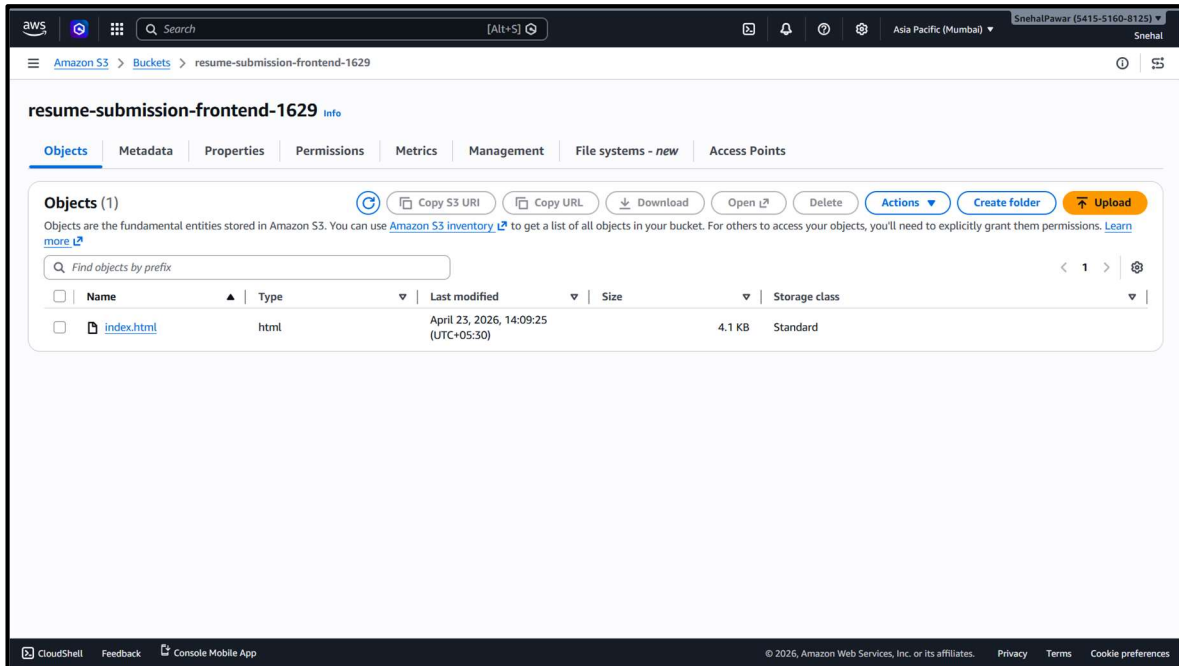
The screenshot displays a web application interface for resume submission. The background is a blue gradient. A white form titled "Resume Submission" is centered. It contains two input fields: "Full Name" and "Email Address". Below these is a file upload section with a dashed border, containing a "Choose File" button and the text "No file chosen". At the bottom of the form is a purple "Submit Resume" button. In the top right corner of the application area, there is a small black box with the text "To exit full screen, press and hold Esc" and an "Esc" key icon.

- **Step 7 — Host Frontend in S3**

Create bucket: resume-submission-frontend

Enable static website hosting

Upload index.html



• Step 8 — Setup CloudFront (Private S3)

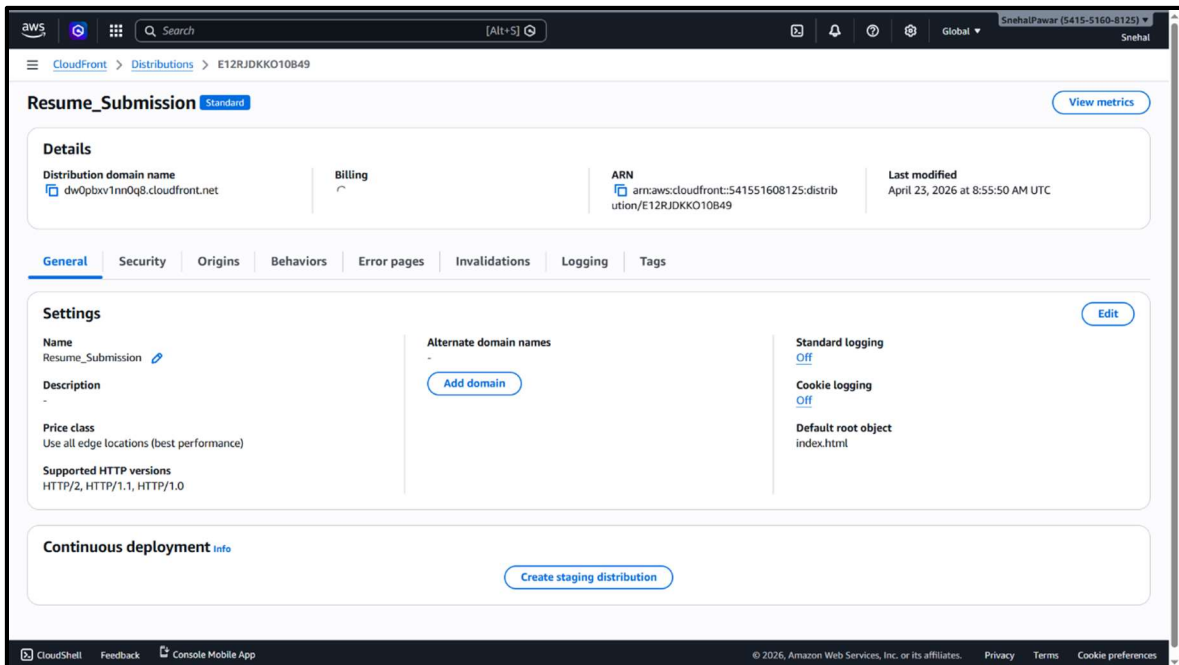
Create distribution

Origin: S3 frontend bucket

Origin access: OAC enabled

Default root object: index.html

Deploy distribution.

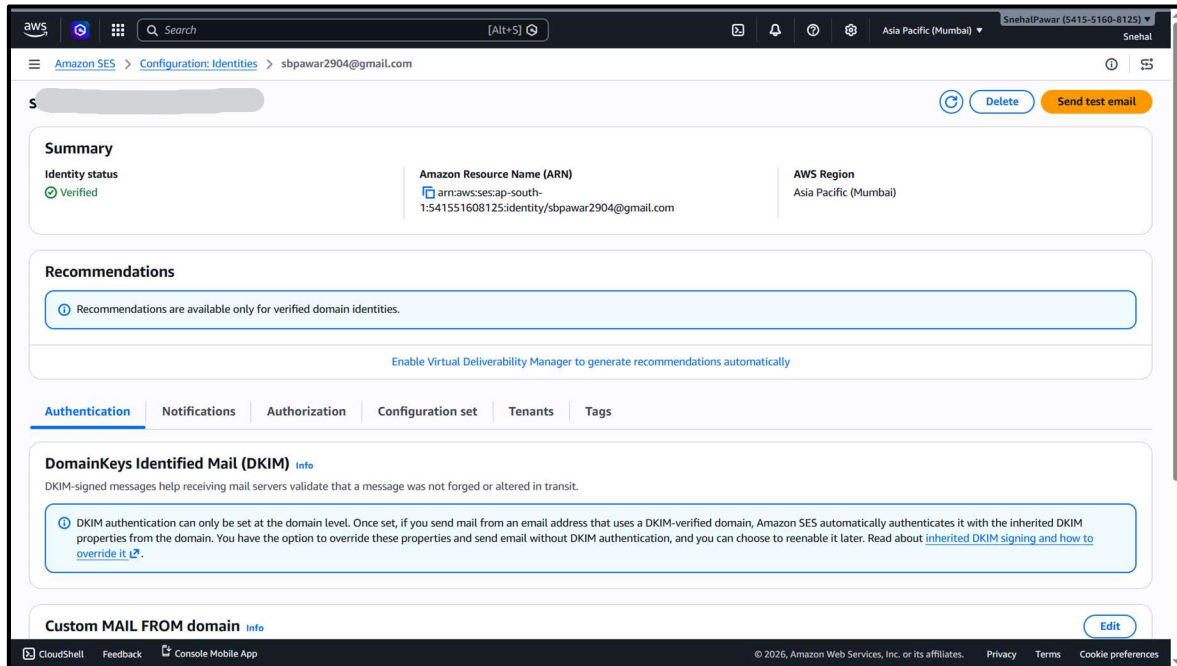


• Step 9 — Configure SES

Verify email identity

Update Lambda to send email

Email sent when resume uploaded.



• Step 10 — Final Flow

User uploads resume

↓

CloudFront serves UI

↓

API Gateway POST request

↓

Lambda executes

↓

Resume stored in S3

↓

Metadata stored in DynamoDB

↓

SES sends notification email

- **Output**

- Resume stored securely
- Metadata tracked
- Email notification triggered
- Fully serverless architecture